A 15-question pre-test for BEAVER dw: which setting is a model ready for?

2026-10-14. Companion to `report.md` / `report_with_figures.md` and `convergence_report.md`. Code and data: `eval/pretest/` (`build_pretest.py`, `score_pretest.py`, `run_pretest.sh`, `pretest_ids.json`, `item_bank.csv`, `calibration.json`, `validation.csv`). How to run it: `eval/pretest/README.md`.

Questions

1. Can a small subset of the 100 dw questions predict a model's full-100 execution accuracy, so a model can be checked before paying for a full run?
2. Which of the current models (claude-opus-5, claude-sonnet-5, claude-haiku-4-5, gpt-5.6-sol/terra/luna) are ready for fewer hints, meaning setting 1 or setting 0?
3. If they aren't, why not?

Short answer

- **15 questions are enough.** Prediction = **0.505 × (correct / 15)**. On 13 runs that were not used to build the pre-test, it lands within **2.2 points** of the full-100 score on average (worst miss 5.8). The ready/not-ready call matched the full run **13/13** times, across settings 0, 1 and 2.
- **All six current models are READY for setting 2.** Opus scores 11/15, Sonnet, Terra and Sol 8/15, and Haiku and Luna 6/15. The pre-test reproduces each one's full score to within 1 point.
- **Settings 1 and 0 are unmeasured for these models.** No current model has been run at either. Going by the older models, Opus, Sonnet, Terra and Sol are **likely ready for setting 1**, Luna and Haiku are **borderline**, and **none is likely ready for setting 0**.
- **Setting 0 fails on schema linking, not SQL writing.** Without hints, the older models picked the right tables about half the time and the right join keys about 15% of the time. These questions need 3–8 joins, so accuracy falls to 2–6% however strong the model is.
- **The pre-test also checks the harness.** Run before the fixes, it would have flagged Haiku's truncation and Sol's HTML-escaped SQL. Those two problems took 3 and 12 points off their full-run scores (see `convergence_report.md` §1 for the escaping).

Method

Data. Dataset `dw`, the 100 `dev_sampled` questions, fewshot baseline. Scores are execution accuracy from `summary_ex_acc_sanitized.json`, which is re-scored with HTML entities unescaped and a bare `None` treated as an empty answer (`eval/sql_sanitize.py`). For claude-haiku-4-5 the 20260901 run is replaced by `…-setting2-maxtok16k-log-20261009`: the 28 questions that had hit `max_tokens = 5120` were regenerated with a 16,384 cap and the same 4,096 thinking budget.

Runs used.

Role	Runs
Calibration (pick the questions, fit the scale factor)	The six 202609XX setting-2 runs: opus-5, sonnet-5, haiku-4-5 (rerun), gpt-5.6-sol/terra/luna

Role	Runs
Holdout (validation only)	gpt-5.4-mini, gpt-5.5, claude-haiku-4-5 (July), claude-sonnet-4-5 at settings 0, 1 and 2, plus qwen2.5-coder-14b at setting 1 (13 runs)

Groups. Each question is put in a group by how many of the six calibration models solved it:

Group	Solved by	Questions in the full 100	In the pre-test
Easy	4–6 of 6	22	6
Medium	2–3 of 6	12	5
Hard	1 of 6	16	3
Unsolved	0 of 6	50	1

Why this mix.

- *Easy-heavy:* at settings 0 and 1, nearly every question any model solves is an easy one, so easy questions carry the readiness signal there.
- *Medium and hard:* these separate the stronger models at setting 2.
- *One unsolved:* half the benchmark is unsolved by everyone and says nothing about current models. One is kept so a future, stronger model shows up as such.

Selection. 3,000 random draws within the group quotas, each required to include ≥ 3 questions whose answer key uses </> (these expose HTML escaping) and ≥ 2 from the longest quartile (these expose truncation). Kept the draw whose prediction best matched the calibration runs. This picked a modestly better set than random: holdout error 2.2 points against a median of 4.7 for random 15-question sets.

Estimator. Predicted full-100 accuracy = c × fraction correct, with c = 0.505 fitted by least squares on the calibration runs, so each question is worth about 3.4 points. This beat a Rasch (IRT) ability model and a group-weighted estimator on holdout error, and it's the easiest to explain.

Verdict.

- **READY:** ≥ 3/15 correct (predicted ≥ 10%) and the health checks pass. The 10% floor sits above the 2–6% every older model scored at setting 0, where models can't be told apart.
- **FIX HARNESS FIRST:** any answer cut off at the token cap, more than 20% of the SQL fails to execute, or any API error.

Results

1. The 15 questions

All 15 ask about MIT's data warehouse: departments, subjects and courses, enrollment, rooms and space, and library materials. Every question ends with "Do not return any rounded answers," which is left off below. *Tables / joins* are counted from the answer key's SQL. *Hint* is the domain-knowledge hint given only at setting 2.

Easy (6)

dw_461: 2 tables, 1 join. Solved by 6/6.

> For each department that is degree granting, has a department budget code greater than 100000, is not the Mathematics department, and is not the Physics department, give the department name and the total clearing cost collector, but only include those departments where the total clearing cost collector is greater than the average clearing cost collector across all such departments.

Filtering, then comparing each department against the average. Hint: Math = `DEPARTMENT_NAME = 'Mathematics'`, Physics = `DEPARTMENT_CODE = '8'`.

dw_19: 3 tables, 3 joins. Solved by 5/6.

> For each department, list the department name, the average, range, and variance of the total units for subjects offered in the 2022 academic year that are degree granting and have at least one subject with more than 9 total units, but only include departments that are not 'Mathematics' (Course 18).

Summary statistics per department with several filters. Hint: Course 18 = Mathematics.

dw_2669: 2 tables, 3 joins. Solved by 6/6.

> For each department, provide the department name, the average, range, and variance of the number of supervisees for all space units associated with that department, and also include the total square footage and the total research volume for each department.

Summary statistics over space and facilities data. No hint.

dw_5166: 2 tables, 1 join. Solved by 5/6.

> For each organization, what is the organization name and the standard deviation (using STDDEV only and never STDDEV_POP) of the area of rooms assigned to it, considering only organizations with more than 5 rooms, and excluding organizations named 'ADMISSIONS'?

Short; tests following an exact function instruction.

dw_663: 4 tables, 3 joins. Solved by 6/6.

> For students whose employee type is 'Student', who are in department 6 (Electrical Engineering and Computer Science), and whose department name is 'Electrical Eng & Computer Sci', show their MIT ID, full name, department name, student year, the running sum of MIT IDs within each student year group ordered by MIT ID, and the coefficient of variation (using STDDEV only and never STDDEV_POP) of the total units for subjects offered in regular academic terms in their department (excluding 'Mathematics').

A running total (window function) plus a coefficient of variation. Easy for the current models; the older models never solved it at settings 0 or 1.

dw_3585: 5 tables, 4 joins. Solved by 5/6.

> For each department, show the department name, the average, range, and variance of the area of rooms assigned to it (considering only rooms with an area greater than 40), as well as the average, range, and variance of the total units for subjects offered in regular academic terms, listing the department name, average area, area range, area variance, average total units, total units range, and total units variance in that order.

Two separate sets of statistics, room area and course units, combined per department.

Medium (5)

dw_4961: 6 tables, 8 joins (the most in the set). Solved by 3/6.

> For each department where the total number of subject enrollments for subjects offered in the current term is greater than 10, and for each course number in that department where the number of enrolled students in the subject summary is greater than 2, provide the department name, the total number of subject enrollments, the course number, the number of unique library subjects offered, the minimum total units for those subjects, and the average number of enrolled students for those subjects. List the department name, total number of subject enrollments, course number, number of unique library subjects offered, minimum total units, and average number of enrolled students in that order.

Filtering at two levels (department, then course), joining enrollment and library data.

dw_140: 3 tables, 3 joins. Solved by 2/6.

> For each department and academic year where the subject enrollment number is greater than 10 and the department is Mathematics, provide the number of unique subjects, the minimum total units offered, and the latest term end date for those subjects.

Looks simple. The difficulty is probably the grain: one row per department *and* academic year.

dw_287: 3 tables, 3 joins. Solved by 2/6.

> For each subject offered in the Mathematics department (Course 18) that has more than 0 enrolled students and is in a degree-granting department, list the department name, subject title, number of enrolled students, and school name, along with a dense rank over the department code ordered by department code, but only include those subjects where the number of enrolled students is greater than the average number of enrolled students for all subjects in the Mathematics department.

A ranking (`DENSE_RANK`) plus a comparison against the department average.

dw_4351: 4 tables, 5 joins, 5 subqueries; a long query. Solved by 2/6.

> For courses offered by the Mathematics department with at least 5 enrolled students, where the number of enrolled students exceeds the average for Mathematics department courses in the same academic year, show the course number, subject title, number of enrolled students, term description, academic year, and the row number within each academic year group ordered by course number. For each such course, also show the name of the degree-granting department, the subject code, the subject code description, and the average and variance of the ASCII values of the course numbers for that subject code, listing the columns in this order: course number, subject title, number of enrolled students, term description, academic year, row number, department name, subject code, subject code description, average ASCII value of course number, and variance of ASCII value of course number.

A comparison against each year's average, a row number, and an unusual ASCII-value statistic. This is one of the questions where Haiku's original run hit the token cap.

dw_777: 5 tables, 2 joins, 4 subqueries. Solved by 2/6.

> List, for each department, the department name, the average and variance of the number of enrolled students in Chemistry courses with at least one associated material record, and the average and

> variance of the total units for subjects offered by degree-granting departments (excluding Biology) in the 2022 academic year. For each row, indicate the reason for inclusion as either 'ChemistryEnrollment' or 'DegreeDeptUnits'. The columns should be: department name, average enrollment, enrollment variance, average total units, variance of total units, and reason for inclusion, in that order.

Two different queries stacked into one result with a label column (`UNION`). Hint: Chemistry = Course 5, Biology = `DEPARTMENT_CODE = '7'`.

Hard (3)

dw_2097: 6 tables, 2 joins. Solved by 1/6.

> For departments where the average number of enrolled students per subject in the 2022FA term is greater than the overall average enrolled students per subject for that term (excluding the Electrical Engineering and Computer Science department and only including departments that grant degrees), list the department name and the average enrolled students per subject.

Older models solved it more often at setting 1 (60%) than current models do at setting 2. The likely trap is whether the exclusions also apply to the "overall average".

dw_5668: 3 tables, 2 joins. Solved by 1/6.

> What is the minimum CIP program code and the total number of departments for courses in the School of Engineering, considering only degree-granting departments, including the corresponding administrative department names, and specifically for the Mechanical Engineering department?

Short but ambiguous: it mixes "School of Engineering" with "specifically Mechanical Engineering". Hint: Mechanical Engineering = Course 2.

dw_4610: 6 tables, 6 joins. Solved by 1/6 (Opus).

> For the top 10 subject titles by the number of reserved library materials for subjects offered in the term '2009SP', what are the subject title, the offering department name, the variance and standard deviation (using STDDEV only and never STDDEV_POP) of the number of enrolled students in subjects offered by that department (considering only departments that grant degrees and subjects with a non-null subject code description), the count of reserved materials, and the total number of units for each subject? Please provide the subject title, the offering department name, the variance, the standard deviation (using STDDEV only and never STDDEV_POP), the count of reserved materials, and the total units, in that order.

A top-10 list, then department-level statistics joined back onto each subject.

Unsolved (1)

dw_4374: 5 tables, 4 joins, 6 subqueries; the longest query. Solved by 0/6, and by none of the 19 runs at any setting.

> List, for each department, the department name, academic year, academic year description, number of degree-granting subjects offered in 2022 (if at least 5), average, range, and variance of total units for those subjects, and the coefficient of variation (using STDDEV only and never STDDEV_POP) of total units for all library reserve materials offered in terms with material status 'U', along with a reason for

> inclusion ('2022 Degree-Granting Subjects' for the first case and 'Library Reserve Materials' for the second). The columns should be: department name, academic year, academic year description, subject count, average total units, total units range, total units variance, coefficient of variation (using STDDEV only and never STDDEV_POP), and reason.

Two different aggregations stacked with a label column, 9 output columns, and conditional inclusion.

What separates the groups.

- Easy questions are mostly one idea: filter, group, summarize, sometimes compare against an average.
- Medium questions add a second structural element: a window function, stacked queries, or filtering at two levels.
- Hard questions are often hard because of ambiguity, not size (dw_2097, dw_5668).
- Every domain-knowledge hint is an MIT department code (Course 18 = Math, 6 = EECS, 2 = Mechanical Engineering, …). Models only get these at setting 2.

2. Current models on the pre-test (setting 2, saved answers)

✓ = correct. Sol and Luna are the unescaped re-scores; Haiku is the 20261009 re-run.

Question	Group	Haiku 4.5	Opus 5	Sonnet 5	Luna	Sol	Terra
dw_461	easy	✓	✓	✓	✓	✓	✓
dw_19	easy	✓	✓	·	✓	✓	✓
dw_2669	easy	✓	✓	✓	✓	✓	✓
dw_5166	easy	✓	✓	✓	·	✓	✓
dw_663	easy	✓	✓	✓	✓	✓	✓
dw_3585	easy	·	✓	✓	✓	✓	✓
dw_4961	medium	·	✓	·	·	✓	✓
dw_140	medium	·	·	·	✓	·	✓
dw_287	medium	·	✓	✓	·	·	·
dw_4351	medium	·	✓	✓	·	·	·
dw_777	medium	·	✓	✓	·	·	·
dw_2097	hard	✓	·	·	·	·	·
dw_5668	hard	·	·	·	·	✓	·
dw_4610	hard	·	✓	·	·	·	·
dw_4374	unsolved	·	·	·	·	·	·
Pre-test		**6/15**	**11/15**	**8/15**	**6/15**	**8/15**	**8/15**
Predicted		20%	37%	27%	20%	27%	27%
Actual (full 100)		19%	37%	28%	20%	27%	27%

Question	Group	Haiku 4.5	Opus 5	Sonnet 5	Luna	Sol	Terra
Verdict		READY	READY	READY	READY	READY	READY

The close match is expected, because these runs chose the questions. Section 4 gives the honest accuracy check.

Before vs after the harness fixes (full runs, setting 2):

Run	Pre-test, original	Pre-test, fixed	Full, original	Full, fixed	Health check on the original
gpt-5.6-sol · 20260901-151849	4/15	8/15	15%	27%	53% of answers HTML-escaped
gpt-5.6-luna · 20260901-140325	5/15	6/15	16%	20%	HTML-escaped answers
claude-haiku-4-5 · 20260901-140932	6/15	6/15	16%	19%	**FIX HARNESS FIRST**: 3/15 cut off at token cap

Opus, Sonnet and Terra were unaffected.

Observations.

- **The medium group is where Opus pulls ahead.** It gets 4 of 5, Sonnet 3, the GPT-5.6 models 1–2 and Haiku 0. The older models score 0–1 there even at setting 2.
- **Same-brand models tend to get the same medium questions right.** Opus and Sonnet are the only models that solve dw_287, dw_4351 and dw_777. Terra and Sol both get dw_4961 (so does Opus). This fits, though it doesn't prove, the same-brand overlap in wrong answers in `convergence_report.md` §3.
- **"Hard" means hard for the current models.** dw_2097 is solved by just one current model at setting 2 but by 60% of old-model runs at setting 1.

3. Who can go down to setting 1 or setting 0?

Not measured yet. No current model has a full run or a pre-test at setting 1 or 0. The estimate below applies the older models' cross-setting ratios: setting-1 accuracy was 0.50–0.77 × setting 2, and setting-0 accuracy was 0.08–0.28 × setting 2.

Model	Setting 2 (measured)	Setting 1 (estimate)	Ready for 1?	Setting 0 (estimate)	Ready for 0?
opus-5	37%	18–28%	likely	3–10%	unlikely, at best borderline
sonnet-5	28%	14–22%	likely	2–8%	unlikely
gpt-5.6-terra	27%	13–21%	likely	2–8%	unlikely

Model	Setting 2 (measured)	Setting 1 (estimate)	Ready for 1?	Setting 0 (estimate)	Ready for 0?
gpt-5.6-sol	27%	13–21%	likely	2–8%	unlikely
gpt-5.6-luna	20%	10–15%	borderline	2–6%	unlikely
claude-haiku-4-5	19%	9–15%	borderline (the July Haiku run scored 9% here: not ready)	2–5%	unlikely

Why setting 0 is a cliff. What each setting puts in the prompt:

Setting	Hints
2	gold tables, column mapping, join keys, domain knowledge, question decomposition
1	gold tables, column mapping, join keys
0	none: the model has to find tables, columns and joins in the schema itself

How the older models did on those sub-steps (F1, from `beaver_results_across_settings_models_20260801.csv`):

	Table F1	Column-mapping F1	Join-key F1	Accuracy
Setting 2	0.90–0.96	0.79–0.83	0.76–0.88	18–26%
Setting 1	0.83–0.95	0.72–0.83	0.68–0.84	9–20%
Setting 0	0.49–0.57	0.38–0.46	**0.13–0.18**	2–6%

- **Setting 2 → 1** removes domain knowledge and decomposition. Accuracy drops by a quarter to a half, but the models still have the right tables and joins.
- **Setting 1 → 0** removes the schema-linking hints, and join-key F1 collapses to about 0.15. These questions need 3–8 joins, and one wrong table or join makes the result wrong.
- **A stronger model doesn't fix this.** gpt-5.5 was the best older model at setting 1 (20%) and the worst at setting 0 (2%). Setting 0 is limited by schema linking, i.e. the retrieval step, more than by the model.

4. Does the pre-test predict? (holdout)

Each of the 13 holdout runs, predicted from its answers on the 15 pre-test questions alone. Group columns are easy / medium / hard / unsolved.

Run	Easy	Med	Hard	Uns	Score	Predicted	Actual	Verdict
haiku-4-5 (July), s0	1/6	0/5	0/3	0/1	1	3%	3%	not ready ✓
sonnet-4-5, s0	1/6	0/5	0/3	0/1	1	3%	6%	not ready ✓
gpt-5.4-mini, s0	0/6	0/5	1/3	0/1	1	3%	5%	not ready ✓

Run	Easy	Med	Hard	Uns	Score	Predicted	Actual	Verdict
gpt-5.5, s0	1/6	0/5	0/3	0/1	1	3%	2%	not ready ✓
haiku-4-5 (July), s1	2/6	0/5	0/3	0/1	2	7%	9%	not ready ✓
qwen2.5-coder-14b, s1	1/6	0/5	0/3	0/1	1	3%	8%	not ready ✓
gpt-5.4-mini, s1	4/6	0/5	1/3	0/1	5	17%	13%	ready ✓
sonnet-4-5, s1	4/6	0/5	2/3	0/1	6	20%	18%	ready ✓
gpt-5.5, s1	5/6	0/5	1/3	0/1	6	20%	20%	ready ✓
gpt-5.4-mini, s2	4/6	1/5	0/3	0/1	5	17%	18%	ready ✓
haiku-4-5 (July), s2	5/6	0/5	1/3	0/1	6	20%	18%	ready ✓
sonnet-4-5, s2	5/6	0/5	2/3	0/1	7	24%	24%	ready ✓
gpt-5.5, s2	6/6	0/5	0/3	0/1	6	20%	26%	ready ✓ (largest miss)

Mean absolute error 2.2 points, worst 5.8. Random 15-question subsets with the same estimator have a median error of 4.7. The verdict agreed 13/13.

The easy group decides readiness at settings 0 and 1. At setting 0 no older model gets more than 1 of the 6 easy questions; at setting 1 the models that turned out ready get 4–5 of 6.

Limitations

- **Small calibration set.** Six models define the groups and the scale factor, and 13 runs from five older models validate them. Treat ±2 points as approximate.
- **Settings 1 and 0 are validated only on older models.** The estimates in section 3 are extrapolations. The first current-model run at setting 1 or 0, pre-test or full, should be checked against them.
- **Ceiling.** 15/15 maps to about 50%. A model clearly stronger than Opus 5 will be under-predicted (the scorer warns at ≥ 13/15). Add its full run and rebuild.
- **Noise.** One question changing is about 3.4 points. A model at exactly 3/15 is borderline.
- **The 10% floor is a judgment call.** It's editable in `eval/pretest/calibration.json`.
- **Scope.** dw only, fewshot baseline. A pre-test predicts only runs made with the same effort and token settings.

Reproduce

```
# rebuild the pre-test from scored runs (no API calls)
cd eval/pretest && ../../beaver-eval/Scripts/python.exe build_pretest.py

# score an existing full run on the 15 questions only (no API calls)
../../beaver-eval/Scripts/python.exe score_pretest.py \
    ../unified-output/fewshot/<run> --summary summary_ex_acc_sanitized.json
```

```
# run the pre-test on a model (15 API calls), e.g. checking readiness for setting
1
PYTHON=../../beaver-eval/Scripts/python.exe ./run_pretest.sh --model claude-opus-5
--setting 1
```

Running the setting-1 pre-test for all six current models is 90 calls, about $5. Adding setting 0 for the models that pass brings it to at most 180 calls, about $15.